Timeline + Build Steps

Hours	Phase	Goal
0-2	Setup and contracts	Lock API contract and event schema. Provision Tiger Data. Claim ElevenLabs promo code.
2-12	Core milestone	Colored bilingual overlay on YouTube, synced to the video. Backend and hypertable stood up in parallel.
12-20	Audio + data loop	TTS in the sidebar, shadowing with scoring, events flowing into Tiger Data, first two dashboard charts.
20-28	Depth and polish	Continuous aggregates, remaining dashboard panels, seed data, compression, live-updating demo moment, edge cases.
28-32	Freeze	Feature freeze. Backup demo video. Devpost writeup.
32-36	Rehearse	Practice the demo. Bugs only, no new features.

Core rule: if the 12-hour milestone slips, drop everything in the Should-have list. A working colored overlay on YouTube beats a half-working everything.

Ownership key

- **Ext** = Extension lead (core overlay, sidebar, shadowing UI)
- **B** = Audio + AI (alignment prompt, ElevenLabs, scoring, Gemini notes)
- **C** = Data (backend, Tiger Data, dashboard)
- **All** = everyone

Phase 1: Setup and contracts (hours 0-2)

Goal: everyone can work in parallel against mocks.

- [All] Pick one target language (Spanish) and one demo video with clean, manual captions.
- [All] Write down the API contract and event schema (see the Project Specs doc). Agree on it before anyone codes.
- [Ext] Scaffold the extension: Manifest V3, TypeScript, Vite with a CRXJS-style plugin for hot reload. Confirm it loads on YouTube.
- [B] Create Gemini API key and ElevenLabs account. Claim the ElevenLabs promo code. Test a Gemini call and one TTS call from a script.
- [C] Provision Tiger Data. Confirm you can connect from a backend. Create the `learning_events` hypertable.
- [All] Create the repo, shared `.env.example`, and a shared channel for blockers.

Done when: extension loads, both APIs respond to a test call, database connects, contract is written.

Phase 2: Core milestone (hours 2-12)

Goal: a caption comes in, Gemini aligns it, and a colored overlay stays in sync with the video. **This is the milestone the whole project depends on.**

Extension track

1. Detect the YouTube video and page navigation (YouTube is a single-page app, so listen for navigation events).
2. Read `ytInitialPlayerResponse` for the caption track URL, then fetch it as `json3` (timestamps included).
3. Log timestamped lines to the console.
4. Hide native captions with CSS. Inject an overlay positioned over the player.
5. Sync to `video.currentTime`: look up the current line, handle pause, seek, and speed changes.
6. Fallback only if step 2 fights you: a `MutationObserver` on YouTube's caption element (no lookahead, so no pre-translation).

Alignment track (Person B)

1. Prototype the alignment prompt in Google AI Studio with structured JSON output.
2. Test on slang, idioms, questions, and fast speech. Iterate until pairs are good.
3. Wrap it as a function: batch of ~30 lines in, aligned JSON out.

4. Translate the first batch immediately so captions start fast, then fill the rest in the background. Cache results.

Backend and data track (Person C)

1. Stand up the backend (FastAPI or Node) with `POST /events` writing to the hypertable.
2. Add `/tts` and `/stt` as stubs returning mock data so the extension can integrate early.
3. Start the dashboard shell with mock data from `GET /stats/...`.

Done when: a normal YouTube video plays with colored bilingual captions in sync. Everyone should be able to see it on their own machine.

Phase 3: Audio and data loop (hours 12-20)

Goal: the click-to-explain, speak, and track loop works end to end.

- [Ext] Sidebar: click a word or phrase to open gloss, grammar note, and example (data comes from the alignment JSON, plus a Gemini call for the example if needed).
- [B] ElevenLabs TTS: word and full-line playback, cached by hash of text + voice + speed. Add slow mode.
- [B] ElevenLabs Scribe: accept recorded audio, return a transcript. Verify Spanish accuracy and latency with a real recording.
- [B] Scoring: normalize both strings, run a word-level diff, return per-word results and an overall score.
- [Ext] Shadowing UI: record button, send audio, render the result using pair colors with missed words underlined in red.
- [Ext] Resolve the microphone-access question (content script versus extension page or offscreen document). If recording is blocked, switch approaches now, not later.
- [C] Real events from the extension (`lookup`, `save`, `shadow_attempt`, `caption_view`) land in Tiger Data.
- [C] Dashboard: first two panels (pronunciation trend and words learned) reading real data.

Done when: you can click a word, hear it, shadow a line, get a score, and see that score appear in the database.

Phase 4: Depth and polish (hours 20-28)

Goal: make both prize stories convincing and the demo reliable.

- [C] Continuous aggregates: daily activity, daily average shadowing score, per-word score history, hourly heatmap data.
- [C] Compression policy on older events.

- [C] Seed script generating a few hundred thousand synthetic events, so the dashboard shows real trends. Label seed data as synthetic in the writeup.
- [C] Remaining panels: trouble words (with a practice button that plays TTS), study heatmap, top videos by new vocabulary.
- [C] **Live update moment:** a fresh shadowing attempt appears in the dashboard chart within a second or two (polling is fine).
- [Ext] Save-to-vocab list.
- [B] Tone and culture notes in the sidebar. Level-adaptive captions if time allows.
- [All] Edge cases: seeking, pausing, videos with no captions, ads, auto-generated captions, very long videos.
- [All] Test on at least 3 different videos.

Done when: the full demo script runs cleanly, twice in a row, without touching the code.

Phase 5: Freeze and package (hours 28-32)

- [All] **Feature freeze.** No new features after this point.
- [All] Record a backup demo video, in case wifi or an API fails during judging.
- [B] Write the Devpost submission. Explicitly name the ElevenLabs products (TTS, Scribe) and Tiger Data features (hypertable, continuous aggregates, compression) used.
- [Ext] Take screenshots and a short GIF for the submission.
- [C] Make sure the dashboard is deployed or runs reliably from a laptop.
- [All] Submit early, then keep polishing.

Phase 6: Rehearse (hours 32-36)

- Run the demo at least 3 times with the timer on.
- Assign speaker roles and decide who drives the screen.
- Prepare a plan B for each failure (video fails, mic fails, API down). See the Demo Guide.
- Fix bugs only. If it works, leave it alone.
- Sleep if you can.

Cut order if time gets tight

Cut in this order:

1. Voice tutor and any other stretch features
2. Gemini weekly summary
3. Heatmap
4. Extra dashboard panels
5. Level-adaptive captions and tone notes

Never cut: the colored overlay, shadowing, or the live dashboard update. Those are the core demo and the two prize moments.

Checkpoints for the team

Ask these out loud at each checkpoint:

- **Hour 4:** Do captions log with timestamps? Does the alignment prompt work standalone?
- **Hour 12:** Is the colored overlay running in sync? If not, what is blocking it?
- **Hour 20:** Does a shadowing attempt reach the database?
- **Hour 28:** Can we run the full demo? What breaks?
- **Hour 32:** Is everything submitted?